

Filtro FIR in Virgola Fissa - Esercitazione 9

Ritornando a quanto detto la scorsa volta, dato un numero reale $x \in \mathbb{R}$, diciamo che la rappresentazione in virgola fissa di x , nel formato $Q_{m,n}$ è quel numero intero X rappresentato su $m+n$ bit ed ottenuto come:

$$X = \text{Trunc}(x * 2^n) \quad \text{oppure} \quad X = \text{Round}(x * 2^n)$$

Convenzionalmente scriviamo che $X \in Q_{m,n}$ e diciamo che il numero reale associato è:

$$\tilde{x} = X/2^n$$

Vediamo ora le operazioni tra numeri a virgola fissa:

Somma e Sottrazione

$$\begin{aligned} x \in \mathbb{R} &\Rightarrow X \in Q_{m,n} \\ y \in \mathbb{R} &\Rightarrow Y \in Q_{m,n} \end{aligned}$$

Da cui la somma / sottrazione:

$$z = x \pm y \in \mathbb{R} \quad \Rightarrow \quad Z = X \pm Y \in Q_{m,n}$$

Infatti se considero:

$$\tilde{x} + \tilde{y} = \frac{X}{2^n} + \frac{Y}{2^n} = \frac{X+Y}{2^n}$$

Esempio:

$$\begin{aligned} x = 2.84 &\Rightarrow X = 91 \in Q_{3,5} \\ y = 0.77 &\Rightarrow Y = 25 \in Q_{3,5} \end{aligned}$$

Da cui:

$$x + y = 3.61 \quad \Rightarrow \quad X + Y = 91 + 25 = 116 \in Q_{3,5}$$

Infatti:

$$\frac{X+Y}{2^5} = \frac{116}{32} = 3,625 \text{ (molto vicino all'originale)}$$

Moltiplicazione

$$\begin{aligned} x \in \mathbb{R} &\Rightarrow X \in Q_{m,n} \\ y \in \mathbb{R} &\Rightarrow Y \in Q_{m,n} \end{aligned}$$

Definisco:

$$w = x \cdot y \in \mathbb{R} \quad \Rightarrow \quad W = X \cdot Y \in Q_{2m,2n}$$

Infatti se considero:

$$\tilde{x} \cdot \tilde{y} = \left(\frac{X}{2^n}\right) \cdot \left(\frac{Y}{2^n}\right) = \frac{X \cdot Y}{2^{2n}}$$

Quindi per la moltiplicazione c'è bisogno di *raddoppiare* il numero di bit per la rappresentazione.

In C servirà usare il formato `int32_t` per rappresentare la moltiplicazione a virgola intera.

Esempio:

$$\begin{aligned} x = 2.84 &\Rightarrow X = 91 \in Q_{3,5} \\ y = 1.18 &\Rightarrow Y = 38 \in Q_{3,5} \end{aligned}$$

Da cui:

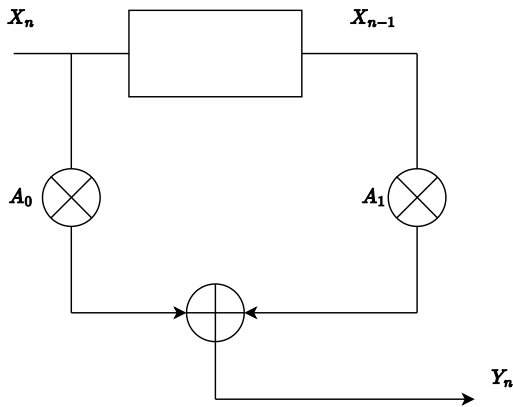
$$w = x \cdot y = 3.3512 \quad \Rightarrow \quad W = X \cdot Y = 3458 \in Q_{6,10}$$

Infatti:

$$\tilde{w} = \frac{3458}{2^{10}} = \frac{3458}{1024} = 3.3769 \text{ (molto vicini all'originale)}$$

Parte 1-2

Prendiamo in esempio un filtro FIR:



Avremmo che:

$$x_n \Rightarrow X_n \in Q_{m,n}$$

$$x_{n-1} \Rightarrow X_{n-1} \in Q_{m,n}$$

$$a_0 \Rightarrow A_0 \in Q_{m,n}$$

$$a_1 \Rightarrow A_1 \in Q_{m,n}$$

Alla fine sommando le varie moltiplicazioni tra X_n e A_n otterrò $Y_n \in Q_{2m,2n}$ per come abbiamo visto prima. Il problema nasce quando si vuole l'uscita a 16 bit (ovvero $Q_{m,n}$) e non a 32 bit (ovvero $Q_{2m,2n}$).

Quindi per mappare l'uscita $Q_{2m,2n}$ ad $Q_{m,n}$ ci sono diversi metodi (nelle slide c'è ne sono 3), quello che trattiamo è il *Riposizionamento Coerente*.

Riposizionamento Coerente

$$X \in Q_{m,n} \quad e \quad Y \in Q_{m,n}$$

Allora:

$$W = X \cdot Y \in Q_{2m,2n}$$

Dove $\tilde{w}$ viene riscritto come:

$$\tilde{w} = \frac{W}{2^{2n}} = \frac{(W \cdot \frac{1}{2^n})}{2^n}$$

Da cui posso dire (come abbiamo visto dividere per 2^n è come fare un shift destro di n bit):

$$\tilde{w} = \frac{W}{2^{2n}} = \frac{(W \cdot \frac{1}{2^n})}{2^n} = \frac{W \gg n}{2^n}$$

Quindi al denominatore ho 2^n e non più 2^{2n} , infatti se dovessi rappresentare il numeratore come $m + n$ bit sono ritornato allo stesso numero di bit usato degli operandi (quindi ritorno al formato iniziale di $Q_{m,n}$).

Quindi l'*algoritmo del riposizionamento coerente*:

$$W \in Q_{2m,2n} \quad e \quad \tilde{w} = \frac{W}{2^{2n}}$$

Genero un nuovo numero intero:

$$W' = [W \gg n]_{n+m}$$

Quindi prendo la rappresentazione binaria di W , la shifto di 5 bit e prendo le 8 cifre significative.

Esempio:

$$x = 2.84 \quad \Rightarrow \quad X = 91 \in Q_{3,5}$$

$$y = 1.18 \quad \Rightarrow \quad Y = 38 \in Q_{3,5}$$

Da cui:

$$w = x \cdot y = 3.3512 \quad \Rightarrow \quad W = X \cdot Y = 3458 \in Q_{6,10}$$

Ora si prende W in binario, faccio uno shift di n bit (nel caso ci fosse lo 0 a sinistra lo faccio rimanere 0, per 1 lascio 1, *non* altero il

segno).

Quindi W in binario:

$$W = 0000110110000010$$

E ora lo shifto di n bit, ovvero 5 bit:

$$W \gg 5 = 0000000001101100$$

Ed ora ottengo W' come: (prendo le prime $m + n$ cifre da destra)

$$W' = [W \gg 5]_8 = 01101100 = 108 \in Q_{3.5}$$

Ed infine ottengo:

$$\bar{w} = \frac{W'}{2^5} = \frac{108}{32} = 3.375$$

Che è molto vicino all'originale a meno di un *errore di quantizzazione*.

Il "coerente" va ad indicare che il formato finale è esattamente $Q_{m,n}$ ovvero quello degli operandi.

Però questo non funziona sempre, infatti funziona quando il formato degli operandi è del tipo:

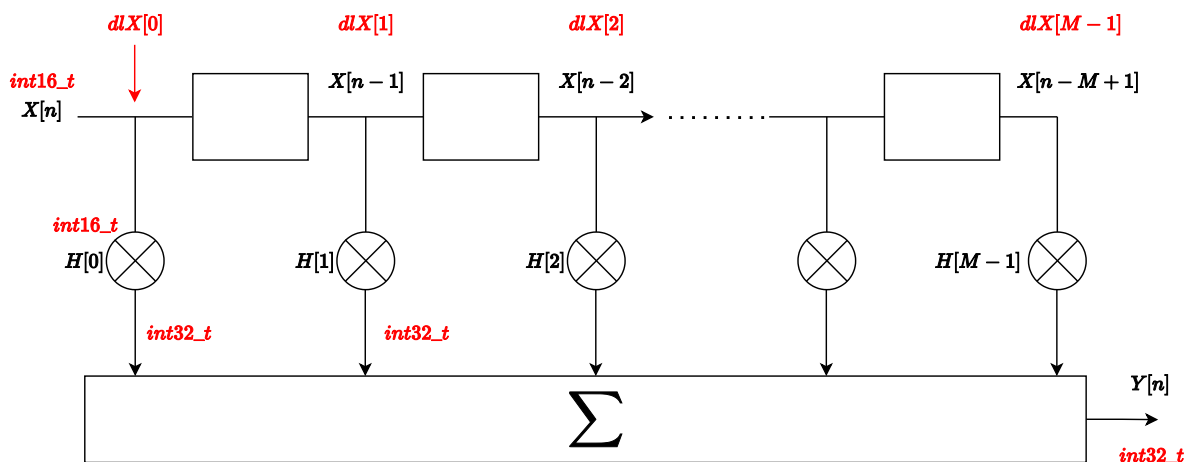
$$Q_{1.(m+n-1)}$$

ad esempio:

$$Q_{1.15}, Q_{1.7}, \dots$$

Però d'altronde questo è il formato che capita più spesso nella realtà, ad esempio nella elaborazione audio abbiamo: 1 bit riservato per la parte intera dei campioni e 15 bit per la parte decimale.

Detto ciò vediamo effettivamente il filtro FIR:



Vediamo, come detto in precedenza, che le uscite dai moltiplicatori sono `int32_t` e che l'uscita totale $Y[n]$ è anche essa a `int32_t`, mentre gli operandi $X[n]$ e $H[M]$ sono `int16_t`.

M sono il numero di moltiplicatori del filtro.

Nella funzione `int16_t FIR(int16_t X, int16_t *H, int n)`, il vettore:

```
static int16_t dX[M] = {0};
```

Non è altro che il vettore che contiene le varie linee di ritardo, usare `static` in questo caso diventa molto utile, infatti facendo ciò al termine della funzione, l'array non viene deallocato (rimane in memoria), alla prossima chiamata della funzione, il registro a scorrimento rimane invariato rispetto alla fine della chiamata scorsa.

Infine il file `FIR_coeff.h` è dove risiedono i coefficienti forniti dall'esercitazione, funziona in maniera simile alle librerie di Python, per includere l'header basta chiamare dentro il file dell'esercitazione:

```
#include "FIR_coeff.h"
```

così avremmo a disposizione tutto il contenuto della "libreria" dentro il file dell'esercitazione.

Come si nota, e come si è studiato, aggiungo un campione alla volta, viene moltiplicato per il coefficiente rispettivo, shifto il registro in modo da far spazio (il singolo campione esce dopo aver passato tutti i coefficienti) ed ritorno il risultato in `int16_t`.

Ci aspettiamo che la sinusoide a 50 Hz passi, invece quella a 250 Hz venga scartata, $\frac{i}{T_s}$ è l'istante di tempo.

Si veda il codice sorgente `exercise1.c` e i plot `exercise1_plot.pdf`.

Homework

L'unica modifica importante al file in questo caso è di usare degli array e processare tutti i campioni in un unico ciclo, invece che processare un singolo campione alla volta e stamparli nel file `.csv` visto che la consegna richiede di effettuare la DFT (o FFT) prima di stampare, e per come è scritta la DFT (o FFT) questa prende tutto il vettore dei campioni invece che un singolo campione per volta.

Si veda il codice sorgente `homework.c` e i plot `homework_plot.pdf`.